Study Help for Midterm 1

ECEn 330 — Introduction to Embedded System Programming — Fall 2026

1. Embedded Systems Overview

- What an embedded system is: a computer inside another device, dedicated to a specific function.
- Moore's Law and why it made embedded computers practical and cheap.
- Where most microprocessors end up (embedded systems, not desktops or servers).
- Typical design constraints: power, size, cost, timing.
- Sensors (physical world → inputs) versus actuators (outputs → physical world).
- Why C is so widely used, and why C alone is not well suited to describing time-ordered behavior.

Practice: Name one sensor and one actuator in a microwave oven, and explain which direction each one converts.

2. C Programming for Embedded Systems

- Fixed-width integer types and their ranges: `int8_t`, `uint8_t`, `int16_t`, `uint16_t`. Pick the smallest type that holds the whole range, including negative values.
- Bit manipulation with masks: set (`|`), clear (`& ~`), toggle (`^`), and test or extract (`(x >> k) & 1`).
- The difference between a bit *number* and a mask *value* (bit 3 has mask `0x08`).
- Masking several buttons at once and comparing against a pattern.
- The conditional operator `cond ? a : b` and the difference between `&` and `&&`.

Practice: Write one C statement that sets bits 0 and 7 of `x` and leaves the other bits unchanged. Then write one that clears them.

3. State Machines (SMs)

- What happens in one *tick*: take one transition (possibly a self-loop), then run the new state's actions once.
- Translating an SM to C: an `enum` of states, a state variable, and a tick function with a transitions switch followed by an actions switch, called repeatedly from `main()`.
- Mealy actions vs Moore Actions
- Tracing an SM against a timing diagram. A plain SM reacts to every input change, because it ticks as fast as it can.

Practice: Explain why a state's action runs again on a tick that takes the state's self-loop.

4. Synchronous State Machines (SynchSMs)

- The period is the time between ticks. The tick function must finish well within one period.
- A synchSM sees its inputs only at tick times. Input changes that come and go between ticks are invisible to it.
- What is a glitch and what are its effects?
- How a timer interrupt (`TimerISR()`) paces the ticks.
- Choosing a period and counter limits so that outputs change at the required rate.

Practice: A synchSM must turn an LED on for 1.5 s and off for 0.5 s, repeatedly. Suggest a period, and give the number of ticks spent in each state.

5. Concurrent SynchSMs and the Task Scheduler

- Each task has its own state diagram, tick function, and period. The tick functions run one after another, not simultaneously.
- How do tasks communicate?
- Does the order of task execution matter? In what circumstances?
- The task scheduler: a timer period equal to the greatest common divisor (GCD) of the task periods, and an array of task structs with `period` and `elapsedTime` fields.

Practice: Describe what the scheduler does to each task on every timer interrupt.

6. Input/Output

- Minimum interevent separation time (MIST).
- Sampling vs period
- Latency
- Choosing a period that meets every constraint
- Switch bounce and debouncing
- Pull-up resistors and floating inputs.

Practice: Why does sampling a switch *faster* not fix bounce?

7. Peripherals

- PWM: duty cycle calculation.
- Serial versus parallel communication.
- ADCs: precision, range, and step size (quantization).

Practice: A 6-bit ADC accepts 0 V to 4 V. What input voltage range maps to output code 0?

8. Labs: Displays and Registers

- Lessons learned
- Reading an ESP32 register description
- Use of C macros
- Good ISR practices.

Practice: Using `GPIO_PIN_REG(n) = 0x3FF44000 + 0x88 + 0x4*n`, find the address of the register for GPIO pin 4.

9. Interrupts

- The path from an IRQ to an ISR.
- How and under what circumstances an ISR function can be called.
- Interrupt priorities and nesting.
- Atomic ISRs - what and how.

Practice: List two things that can go wrong when an ISR takes too long.

10. Hardware Interfacing

- From the in class discussions - What are the issues with long wires?
- Types of system oscillators: cost versus accuracy and stability.
- Voltage levels: use of level shifters, voltage dividers, and buffers.

Practice: Thought Question: Why does a crystal oscillator matter for a UART link?

11. AI and Engineering Practice

- Use of AI in the workplace
- The engineer remains responsible for reviewing and verifying everything an AI tool produces.
- When are humans ultimately responsible for produced code?